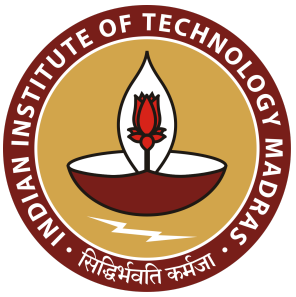
GRAND CHALLENGE - 2025

TOOLS INSTALLATION IN UBUNTU



DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

0.1 Proprietary Notice

Copyright © 2025, **SHAKTI@IIT Madras**.All rights reserved.
Information in this document is provided “as is,” with every effort ensuring that the documentation is as accurate as possible.

SHAKTI@IIT Madras expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SHAKTI@IIT Madras does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SHAKTI@IIT Madras reserves the right to make changes without further notice to any products herein.

The project was funded by Ministry of Electronics and Information Technology (MeitY), Government of India

0.2 Release Information

Version	Date	Updates	Reviewed by
1.0	16/09/2025	Vignesh K	Kotteeswaran E
Draft V.1	07/10/2025	Vignesh K	Kotteeswaran E

TABLE OF CONTENTS:

0.1 Proprietary Notice..... 2
0.2 Release Information..... 3
1. Introduction:..... 5
2. Common Dependencies:..... 5
3. Hardware dependencies..... 5
4. Installing Bluespec complier:..... 13
5. Installing GTKWAVE:..... 13
6. Software Dependencies:..... 14
7. Installing SPIKE:..... 15
8. Installing VERILATOR:..... 16
9. Installing GTKTerm..... 17

1. Introduction:

This document details the hardware and software tools being needed for Shakti SoC development. The hardware setup with Ubuntu 22.04 needs the following .

- Vivado IDE
- Vivado Cable driver.
- Copying board files.
- Bluespec Compiler.
- GTKWAVE

The software dependencies are

- RISC-V GNU toolchain.
- Spike
- Verilator.
- Gtkterm

2. Common Dependencies:

Install these dependencies which are common in some of them.

```
sudo apt-get install autoconf automake autotools-dev curl make libmpc-dev libmpfr-dev libgmp-dev bc  gawk build-essential bison flex texinfo gperf libtool patchutils zlib1g-dev pkg-config
```

3. Hardware dependencies

Install the following dependencies before proceeding with any other installation. Copy the table 1 and paste the same in a terminal.

3.1. Install dependencies

Install the following dependencies by running it in the ubuntu terminal.

```
sudo apt-get install libusb-1.0-0-dev libusb-0.1-4 libftdi1 libftdi1-2 libncurses5
```

3.2. Vivado Setup:

a. Download Vivado SDK edition from the AMD website.

```
https://www.xilinx.com/member/forms/download/xef-vivado.html?filename=Xilinx_Vivado_SDK_2018.3_1207_2324.tar.gz
```

A login is required to download the tar file in the AMD website.

b. Untar the file “<filename>.tar.xz”

```
tar xf Xilinx_Vivado_SDK_2018.3_1207_2324.tar.gz
```

c. Give write permission to the file “xsetup” by using the command

```
chmod 777 xsetup
```

d. Run the code “./xsetup” in the terminal”

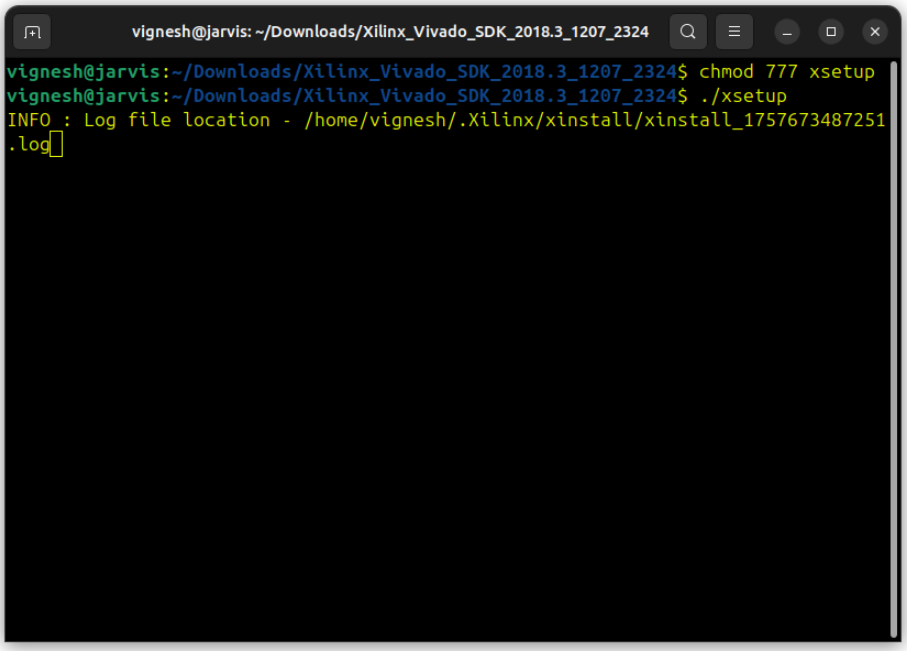


Figure 1: Installation Setup command

Installation flow:

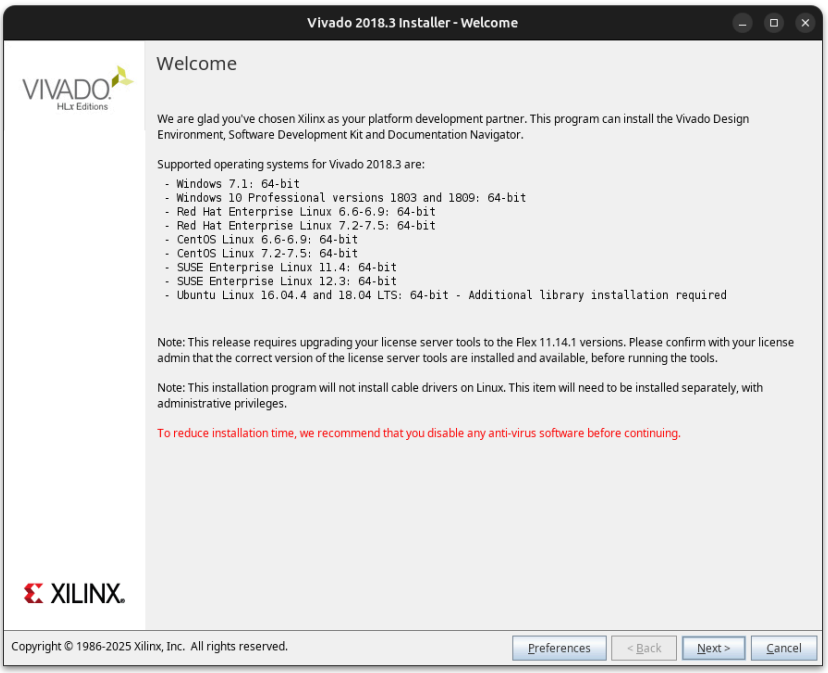


Figure 2:Give next and proceed.

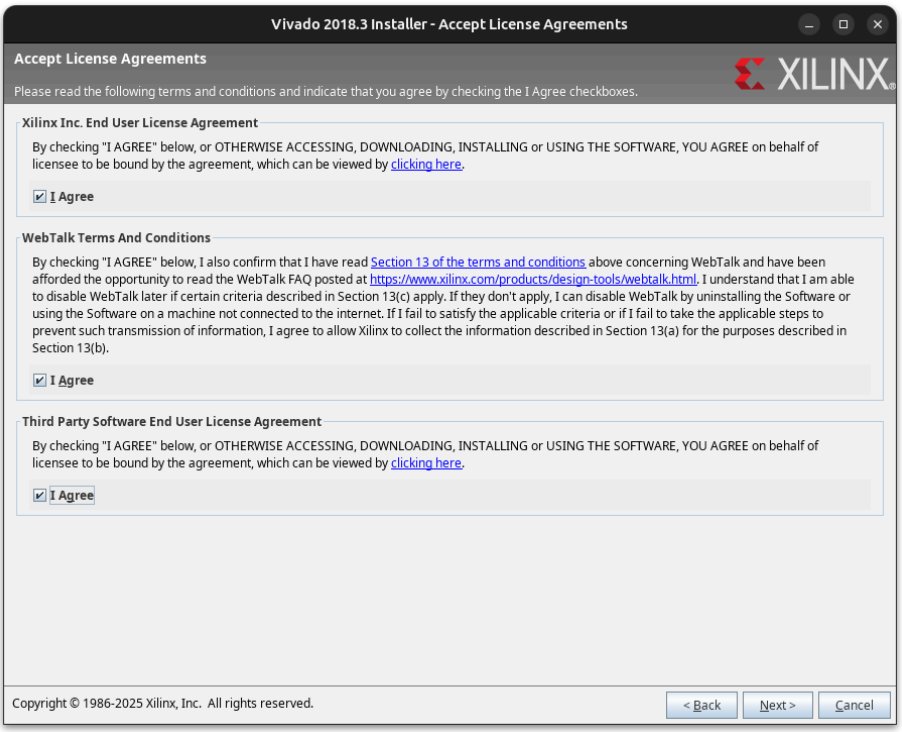


Figure 3: Agree to everything and proceed.

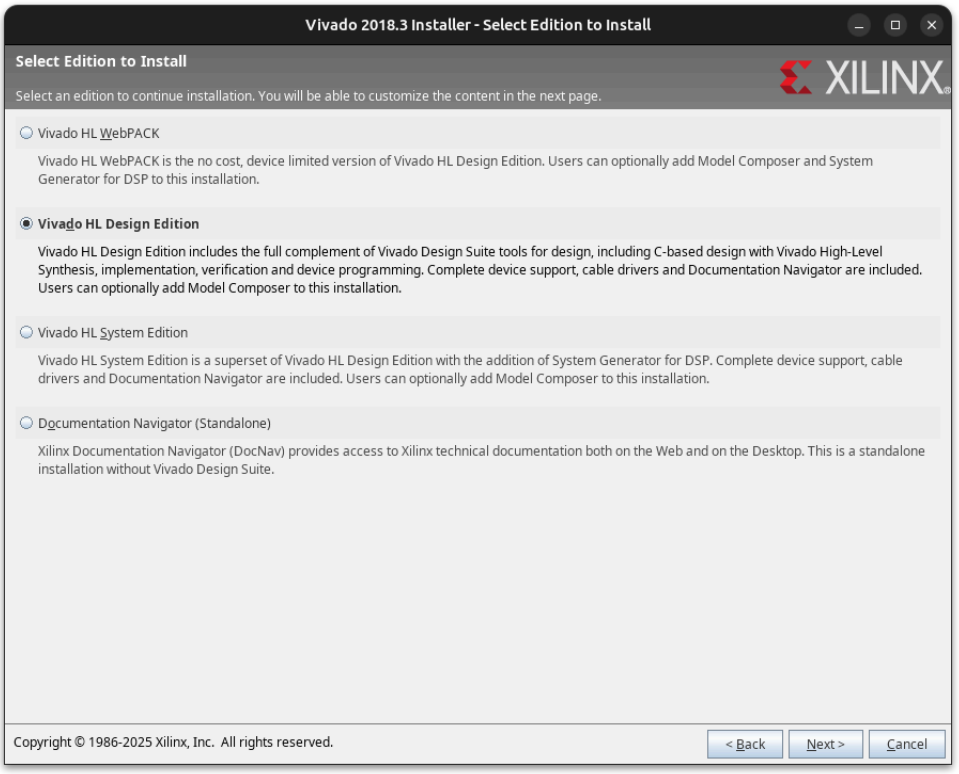


Figure 4: Select Vivado HL Design edition and proceed.

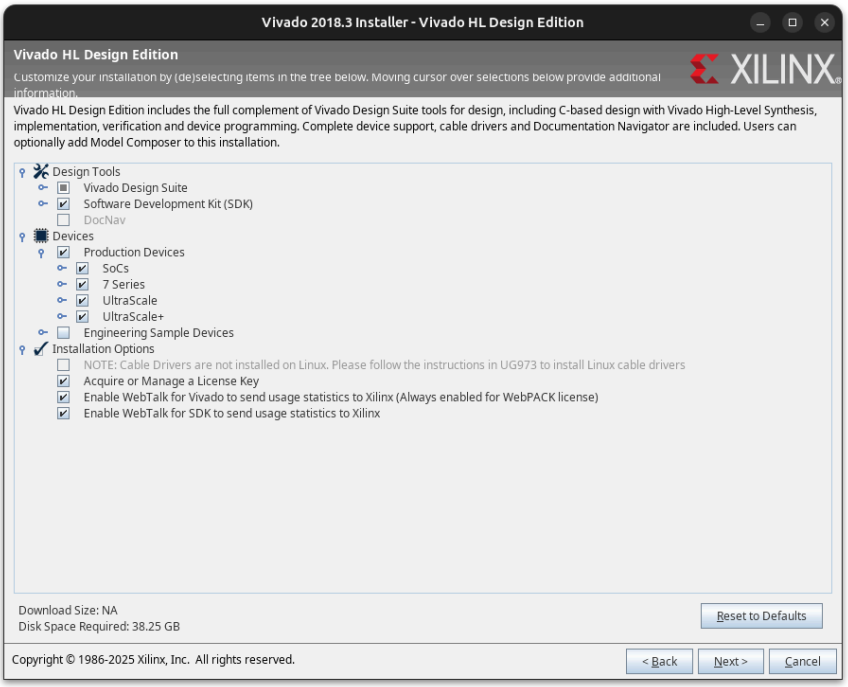


Figure 5: Select Everything and proceed.

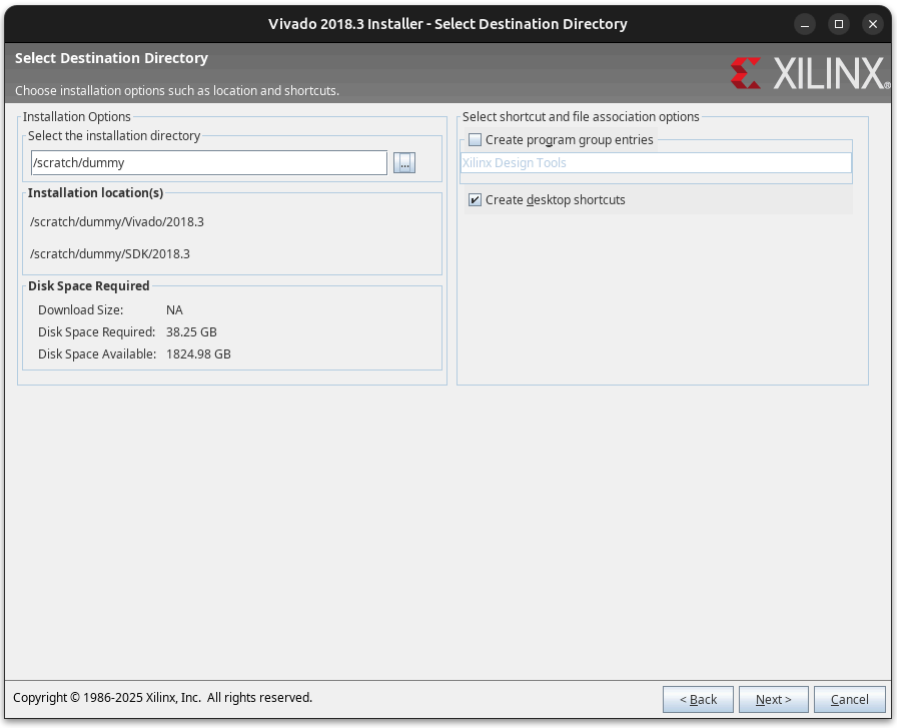


Figure 6: Enter the directory wherever Vivado is to be installed and deselect this create program group entries box and then proceed.

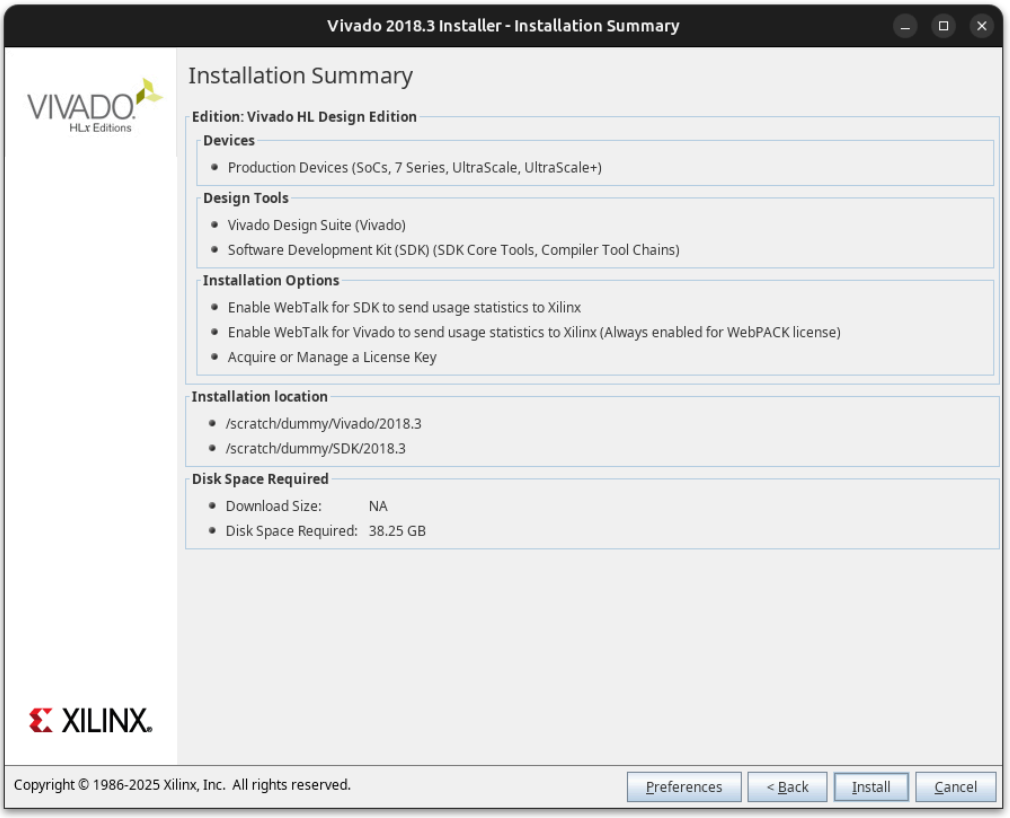


Figure 7: At this stage, install and proceed.

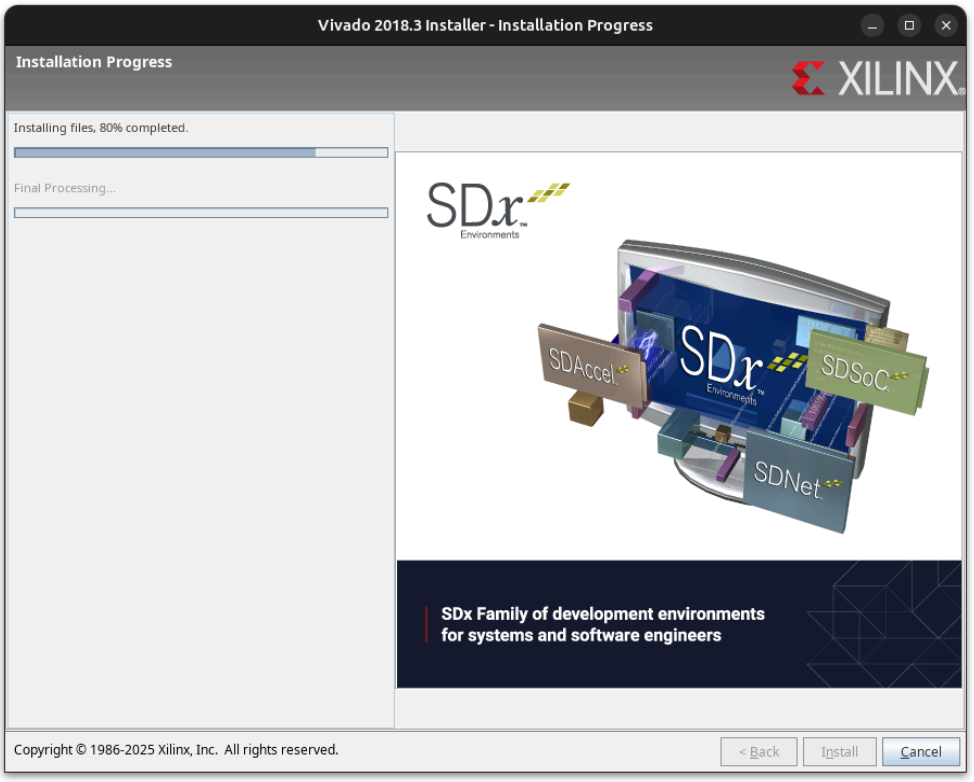


Figure 8: Vivado Installation Window

e. After installation, do the following.

- Go to the folder "[installed folder]/vivado/Vivado/2018.3/data/xicom/cable_drivers/lin64/install_script/install_drivers/"
- run "sudo ./install_drivers"

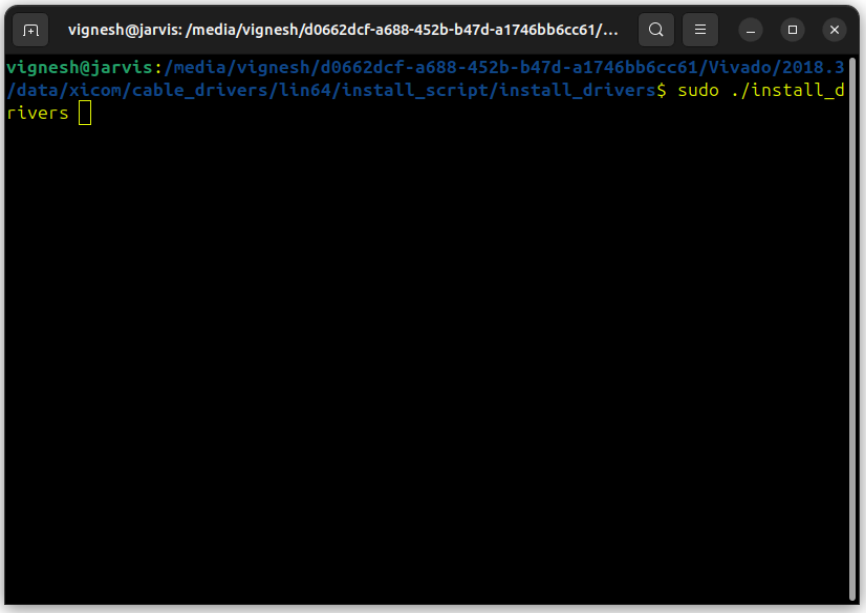


Figure 9: Drivers Installation command

These drivers are not available pre-installed in the Vivado ,so it is required to install them before connecting the board.

3.3. Adding board files

a. Board files can be added on cloning the board files from digilent's github page.

b. In this GitHub repository, there are board files for almost every Digilent Boards.

```
git clone https://github.com/Digilent/vivado-boards/tree/master/new/board_files
```

c. Add the cloned board files from Github which is required to /path/to/Vivado/2018.3/data/boards/board_files.

3.4. Invoking Vivado application

Vivado application can be invoked in two following ways.

1.Through icon in home:

- 1. Go to the Desktop and find the vivado icon
- 2. Right click and select "Allow Launch"
- 3.Double clicking the icon should open Vivado



Figure 10: Opening Vivado through icon

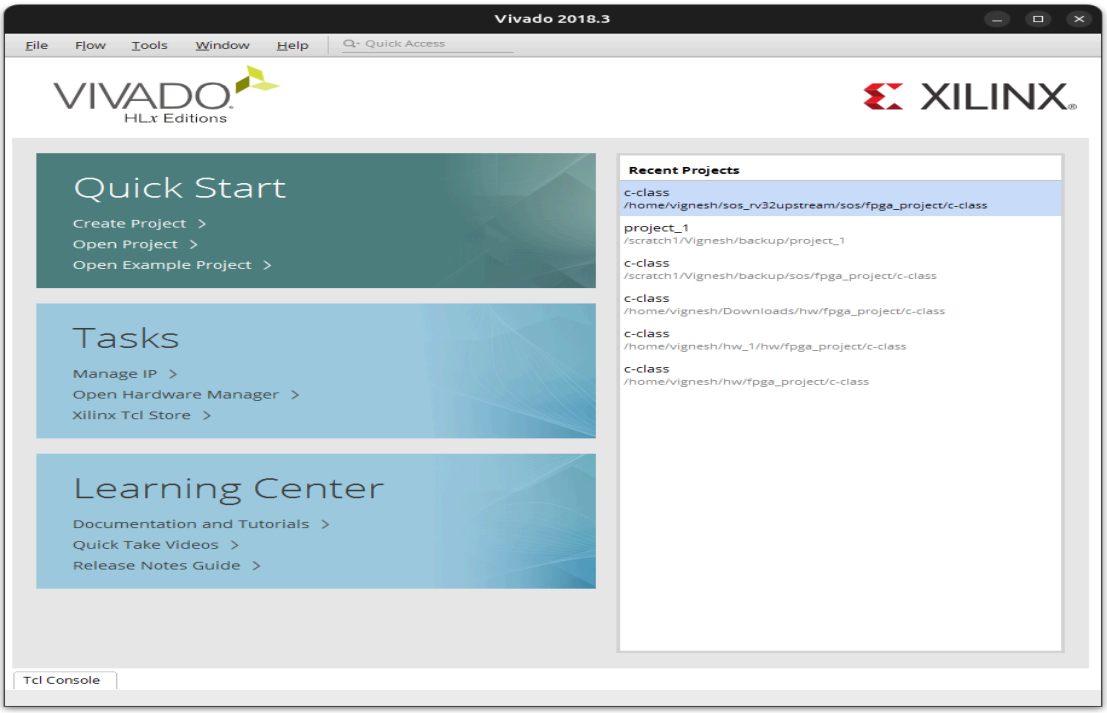


Figure 11: Opening Vivado

2.Through Terminal

- Go to following path “/path/install/vivado/Vivado/2018.3/” Right click and open new terminal
- Run the following command ./vivado , this should open vivado

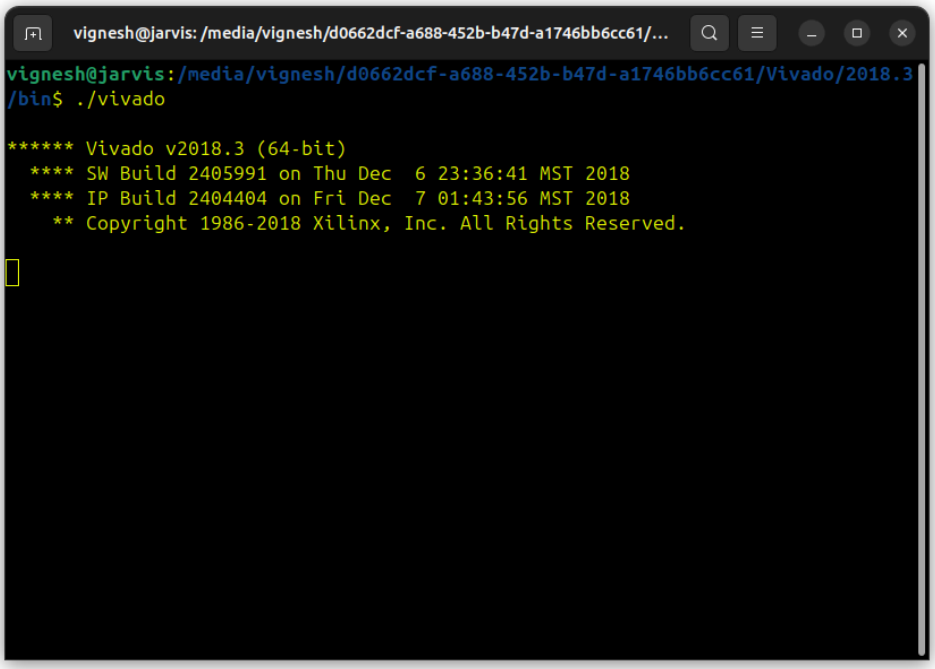


Figure 12: Opening Vivado through terminal

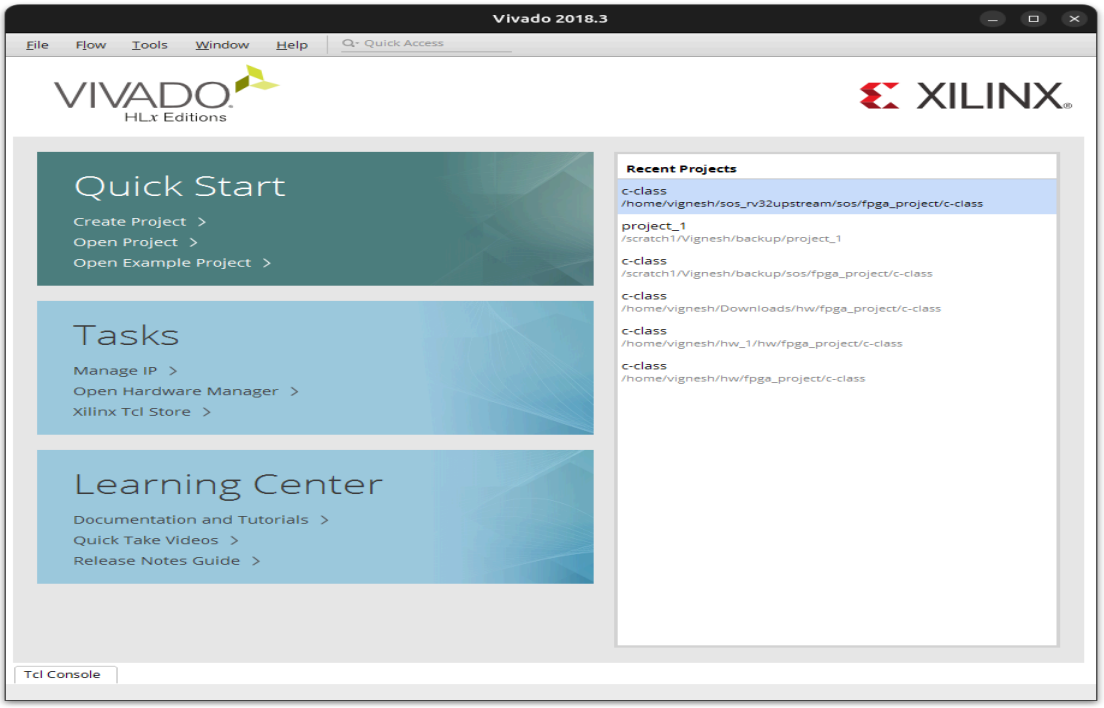


Figure 13: Opening Vivado

4. Installing Bluespec complier:

4.1. Install the dependencies:

```
sudo apt-get install ghc libghc-regex-compat-dev libghc-syb-dev
libghc-old-time-dev libghc-split-dev tcl-dev
```

4.2. Clone this repository:

```
git clone --recursive https://github.com/B-Lang-org/bsc
```

4.3. Change directory to bsc and checkout to 2025.01.1 tag.

```
cd bsc
git checkout 2025.01.1
```

4.4. Install Bluespec compiler from source:

```
sudo make install-src
```

4.5. Add BSC to PATH with this command:

```
echo 'export PATH=/home/test/bsc/inst/bin:$PATH' >> ~/.bashrc
bash
```

4.6. It can be invoked by these commands:

```
bsc
```

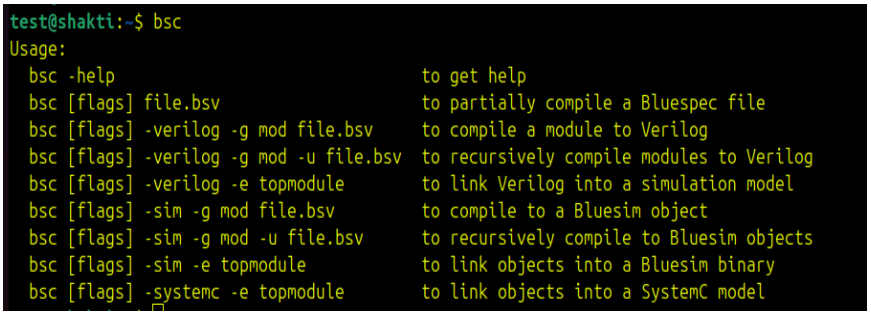


Figure 14: Invoking Bluspec Compiler

5. Installing GTKWAVE:

5.1. Install gtkterm using the command:

```
sudo apt install gtkwave -y
```

6. Software Dependencies:

6.1. Install dependencies.

```
sudo apt-get install  python3 python3-pip libexpat-dev ninja-build git cmake
libglib2.0-dev libslirp-dev
```

6.2. Install RISCV-GNU-TOOLCHAIN:

```
git clone https://github.com/riscv/riscv-gnu-toolchain
```

6.3. After cloning go into the folder , update the submodule, change version of gcc to 13.2 ,configure and make.

```
cd riscv-gnu-toolchain
git submodule update --init --recursive
sudo rm -rf gcc
git clone https://gcc.gnu.org/git/gcc.git
cd gcc
git checkout releases/gcc-13.2.0
cd ..
mkdir build_rv
./configure --prefix=/opt/riscv/build_rv --enable-multilib
sudo make -j $(nproc)
```

These commands will configure ,make and install the toolchain. This will contain both 32-bit and 64-bit toolchain

6.4. Add path to .bashrc with the following commands:

(This will by default invoke the toolchain)

```
echo 'export PATH='[Path to risc-v
toolchain]/riscv-gnu-toolchain/build_rv/bin:$PATH' >> ~/.bashrc
bash
```

6.5. To verify try these commands:

```
which riscv64-unknown-elf-gcc

riscv64-unknown-elf-gcc --print-multi-lib
```

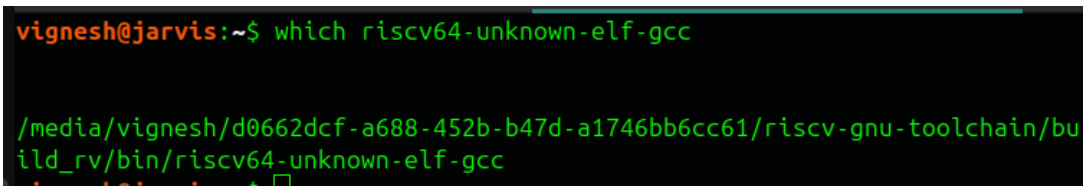


Figure 15: Checking the path of multilib RISCV toolchain

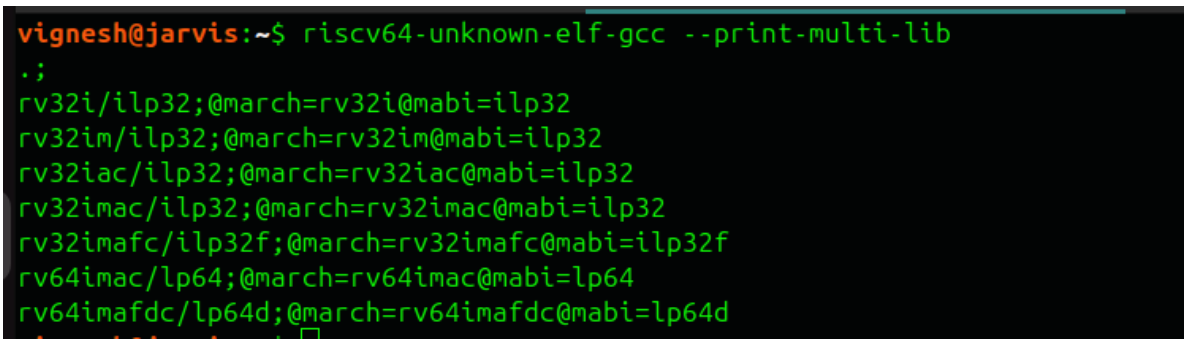


Figure 16: Checking the invocation of all other toolchain

7. Installing SPIKE:

7.1. Install Dependencies:

```
sudo apt-get install device-tree-compiler libboost-regex-dev libboost-system-dev
```

7.2. Clone the RISCV_ISA_SIM repository:

```
git clone https://github.com/riscv-software-src/riscv-isa-sim.git
```

7.3. Go into the cloned directory and checkout to the given git.

```
cd riscv-isa-sim
git checkout d39cbb3ad27b2de882260c0a22f11a59efebb151
```

This will checkout the repository to the specified branch.

7.4. Create a directory named build, go inside, configure and make.

```
mkdir build
cd build
export RISCV=/path/to/install/spike
./configure --prefix=$RISCV
sudo make -j $(nproc)
sudo make install
```

These commands will configure ,make and install spike.

7.5. Spike can be invoked by giving these commands:

```
./spike
```

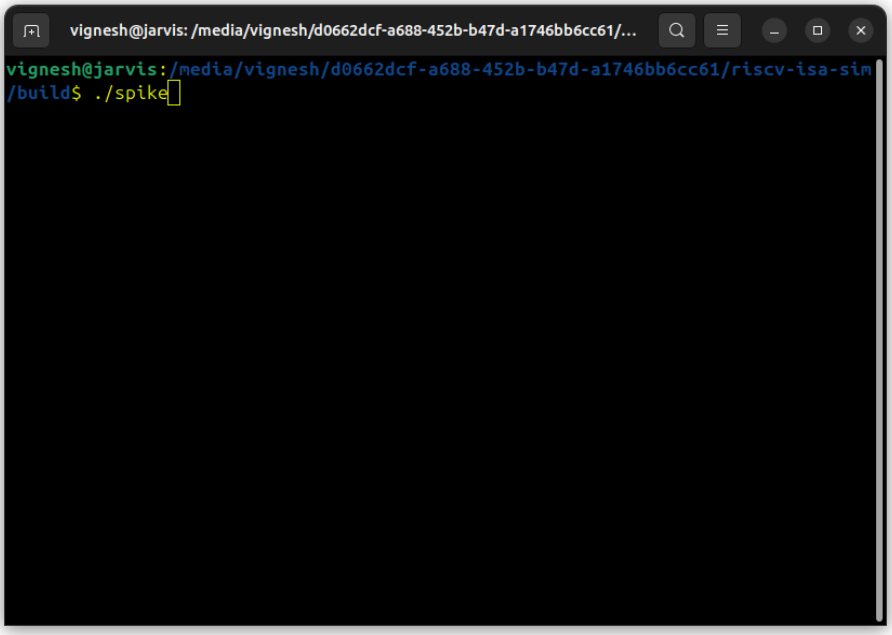


Figure 17: Invoking Spike

8. Installing VERILATOR:

8.1. Install the dependencies:

```
sudo apt-get install perl g++ ccache libgoogle-perftools-dev numactl perl-doc libfl2
libfl-dev zlib1g help2man
```

8.2. Clone this repository:

```
git clone https://github.com/verilator/verilator
```

8.3. Change directory to bsc and checkout to 2025.01.1 tag.

```
cd verilator
git checkout v5.024
```

This will checkout to the branch v5.024

8.4. Install Verilator:

```
unset VERILATOR_ROOT
autoconf
./configure
make -j `nproc`
sudo make install
```

These commands will configure ,make and install verilator.

8.5. It can be invoked by these commands:

```
verilator
```

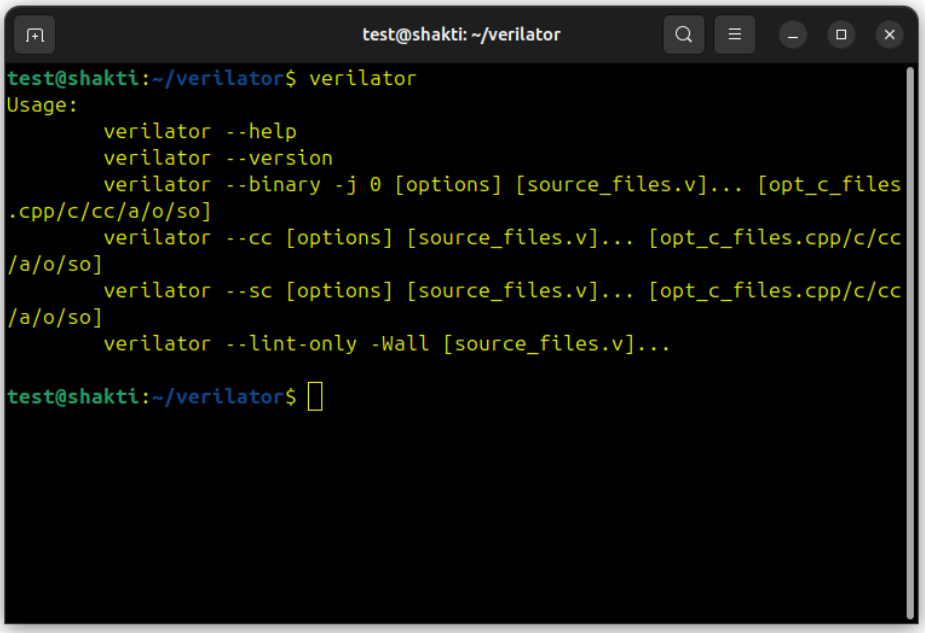


Figure 18: Checking Verilator through command

9. Installing GTKTerm

9.1. Install gtkterm using the command:

```
sudo apt install gtkterm -y
```

9.2. Opening gtkterm:

```
sudo gtkterm
```

This completes the tools setup for both hardware build and software build.